

Национальный исследовательский университет ИТМО

Факультет безопасности информационных технологий

Алгоритмы и структуры данных

Лабораторная работа №1

«Цифровая сортировка чисел из файла»

Выполнил:

Студент группы N3247

Сивоконь А.А.

(подпись)

Преподаватель:

Ерофеев С.А.

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026

Содержание

1	Постановка задачи	2
2	Описание алгоритма сортировки	3
2.1	Алгоритм цифровой сортировки (Radix Sort LSD)	3
2.2	Маршрут программы	4
2.3	Структуры хранения данных	5
3	Описание используемых переменных	6
4	Тестирование программы	8
4.1	Тест 1: статический массив (успешная сортировка)	8
4.2	Тест 2: динамический массив	8
4.3	Тест 3: вектор (авторасширяемый массив)	9
4.4	Тест 4: несуществующий файл	10
4.5	Тест 5: пустой файл	10
5	Блок-схемы алгоритма	11
6	Исходный код программы	12
6.1	Файл main.c	12
6.2	Файл radix_sort.h	14
6.3	Файл radix_sort.c	16
	Заключение	24
	Приложение А. Блок-схемы алгоритма	25

1. Постановка задачи

Цель работы: разработать программу цифровой сортировки (Radix Sort) для последовательности целых неотрицательных чисел, считываемых из файла, которая записывается в статический массив, динамический массив или вектор по выбору пользователя; определить сложность программы.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать чтение массива чисел из файла;
2. Предусмотреть возможность выбора между тремя структурами хранения данных;
3. Реализовать алгоритм цифровой сортировки LSD (от младшего разряда к старшему);
4. Реализовать функцию вывода массива на экран;
5. Оценить теоретическую сложность алгоритма;
6. Провести тестирование программы на различных наборах данных;
7. Составить блок-схемы алгоритма и оформить отчёт.

2. Описание алгоритма сортировки

2.1. Алгоритм цифровой сортировки (Radix Sort LSD)

Цифровая сортировка — нерекурсивный алгоритм, обрабатывающий числа поразрядно. В варианте LSD обработка ведётся от *младшего* разряда к *старшему*.

Пошаговый алгоритм:

1. **Нахождение максимального элемента.**

Функция `find_max` обходит весь массив и возвращает наибольшее значение. По нему определяется число разрядов d .

2. **Цикл по разрядам** ($\text{exp} = 1, 10, 100, \dots$).

Пока $\text{max} / \text{exp} > 0$, выполняется сортировка подсчётом по текущему разряду.

3. **Один проход сортировки подсчётом** (`counting_sort_by_digit`):

- (a) Обнулить счётчики `count[0..9]`.
- (b) Для каждого элемента вычислить цифру разряда: $(\text{arr}[i] / \text{exp}) \% 10$; увеличить соответствующий счётчик.
- (c) Накопить префиксные суммы: `count[i] += count[i-1]`.
- (d) Обходя массив *справа налево*, разместить элементы в `output[]` — обратный обход гарантирует устойчивость сортировки.
- (e) Скопировать `output[]` обратно в `arr[]`.

Оценка сложности алгоритма:

- Временная: $O(d \cdot (n + k))$, где n — количество элементов, d — число разрядов в максимальном числе, $k = 10$ — основание счисления. При фиксированных d и k сложность **линейная**: $O(n)$.
- Пространственная: $O(n + k)$ — вспомогательный массив `output[n]` и счётчики `count[k]`.

Точный подсчёт с учётом всех 7 типов операций (вызов метода, выход из метода, индексация массива, переход по ссылке, присваивание, сравнение, арифметика):

$$T_{find_max} = 6n - 1 \quad ()$$

Разбивка одного прохода `counting_sort_by_digit` по шагам ($k = 10$):

Шаг	Операций
Подготовка (malloc + проверка)	4
1. Обнуление <code>count[0..k-1]</code>	$4k + 2 = 42$
2. Подсчёт вхождений	$8n + 2$
3. Префиксные суммы	$7k - 5 = 65$
4. Расстановка в <code>output[]</code>	$13n + 3$
5. Копирование обратно	$5n + 2$
free + выход	2
Итого	$26n + 11k + 10$

$$T_1 = 26n + 11k + 10 \quad (k=10 : 26n + 120)$$

$$T_{radix_sort} = \underbrace{(6n - 1)}_{find_max} + \underbrace{(7 + 5d)}_{\text{}} + d \cdot (26n + 11k + 10) = 6n + 6 + d \cdot (26n + 11k + 15)$$

При $k = 10$: $T = 6n + 6 + d \cdot (26n + 125)$, что асимптотически соответствует $O(d(n + k))$.

2.2. Маршрут программы

Алгоритм работы программы представляет собой следующую последовательность шагов:

1. **Вывод меню выбора структуры данных:** пользователь вводит 1, 2 или 3.
2. **Инициализация трёх структур:** `static_init`, `dynamic_init`, `vector_init`.

3. **Чтение чисел из файла** (`read_file`):

- первый проход: подсчёт количества чисел n ;
- второй проход: считывание чисел в выбранную структуру.

4. **Получение указателя** на массив и его размер (`get_array_ptr`).

5. **Вывод исходного массива** на экран (`print_array`).

6. **Выполнение Radix Sort** (`radix_sort`).

7. **Вывод отсортированного массива** на экран.

8. **Вывод оценки сложности:** n , максимальный элемент, d , k , число операций, число дополнительных ячеек памяти.

9. **Освобождение памяти:** `free` вызывается для динамического массива и вектора.

2.3. Структуры хранения данных

Программа поддерживает три способа хранения входного массива:

1. **Статический массив** (`StaticArray`) — буфер `int data[1000]` на стеке. Размер ограничен константой `STATIC_MAX = 1000`.
2. **Динамический массив** (`DynamicArray`) — блок памяти, выделяемый через `malloc` один раз после подсчёта числа элементов. Не требует перевыделения.
3. **Вектор** (`Vector`) — авторасширяемый массив: начальная ёмкость 4, при заполнении удваивается через `realloc`. Стратегия удвоения даёт $O(1)$ амортизированного времени на добавление.

3. Описание используемых переменных

В таблице 1 приведено описание переменных программы.

Таблица 1 – Описание переменных программы

Переменная	Тип	Границы	Назначение
Главная программа (main.c)			
mode	int	{1, 2, 3}	Выбор структуры данных
sa	StaticArray	—	Статический массив (стек)
da	DynamicArray	—	Динамический массив (куча)
v	Vector	—	Авторасширяемый вектор
n	int	$[1; 2^{31} - 1]$	Количество прочитанных чисел
arr	int*	адрес	Указатель на данные массива
sz	int	$[1; 2^{31} - 1]$	Размер массива
max_val	int	$[0; 2^{31} - 1]$	Максимальный элемент массива
d	int	$[1; 10]$	Число разрядов max_val
tmp	int	$[1; 2^{31} - 1]$	Временная для подсчёта разрядов
Структуры хранения (radix_sort.c)			
data[1000]	int [1000]	$[0; 2^{31} - 1]$	Буфер статического массива
size	int	$[0; 1000] / [0; n]$	Фактическое число элементов
capacity	int	{0, 4, 8, 16, ...}	Выделенная ёмкость вектора

Переменная	Тип	Границы	Назначение
new_cap	int	$[4; 2^{31} - 1]$	Новая ёмкость при расширении
tmp	int*	адрес	Результат <code>realloc</code>
Ввод/вывод (radix_sort.c)			
fp	FILE*	адрес	Указатель на открытый файл
n	int	$[1; 2^{31} - 1]$	Счётчик при первом проходе
tmp	int	$[0; 2^{31} - 1]$	Значение при счёте чисел
val	int	$[0; 2^{31} - 1]$	Считанное значение из файла
i	int	$[0; n - 1]$	Индекс цикла второго прохода
Цифровая сортировка (radix_sort.c)			
exp	int	$\{1, 10, \dots, 10^{d-1}\}$	Текущий разряд
output	int*	адрес	Вспомогательный массив размером n
count[10]	int [10]	$[0; n]$	Счётчики для цифр 0–9
digit	int	$[0; 9]$	Цифра текущего разряда
i	int	$[0; n - 1]$	Индекс цикла
max	int	$[0; 2^{31} - 1]$	Текущий максимум в <code>find_max</code>

4. Тестирование программы

Для тестирования использовался файл `input.txt`:

```
170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5
```

4.1. Тест 1: статический массив (успешная сортировка)

```
+-----+
| Цифровая сортировка (Radix Sort) |
+=====+
| Выберите структуру данных:      |
| 1. Статический массив           |
| 2. Динамический массив          |
| 3. Вектор (авторасширяемый)     |
+-----+
Ваш выбор: 1

Чтение файла 'input.txt'...
Прочитано 20 чисел.

Исходный массив:
170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5

Выполняем Radix Sort...
Отсортированный массив:
1 2 5 7 23 24 40 45 66 75 88 90 125 170 312 377 456 802 999 1000

Оценка сложности
n (элементов)      20
max элемент        1000
d (разрядов)       4
k (основание, BASE) 10
Операций:          2706
Доп. ячеек:        30
```

Массив успешно отсортирован. Данные хранились в буфере `int data[1000]` на стеке, память на куче не выделялась.

4.2. Тест 2: динамический массив

Те же входные данные, выбор 2:

Ваш выбор: 2

Чтение файла 'input.txt'...

Прочитано 20 чисел.

Исходный массив:

170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5

Выполняем Radix Sort...

Отсортированный массив:

1 2 5 7 23 24 40 45 66 75 88 90 125 170 312 377 456 802 999 1000

Оценка сложности

n (элементов)	20
max элемент	1000
d (разрядов)	4
k (основание, BASE)	10
Операций:	2706
Доп. ячеек:	30

Программа корректно выделила ровно 20 ячеек через `malloc` и освободила память через `free`.

4.3. Тест 3: вектор (авторасширяемый массив)

Те же входные данные, выбор 3:

Ваш выбор: 3

Чтение файла 'input.txt'...

Прочитано 20 чисел.

Исходный массив:

170 45 75 90 802 24 2 66 125 377 40 1 999 312 7 88 456 23 1000 5

Выполняем Radix Sort...

Отсортированный массив:

1 2 5 7 23 24 40 45 66 75 88 90 125 170 312 377 456 802 999 1000

Оценка сложности

n (элементов)	20
max элемент	1000
d (разрядов)	4
k (основание, BASE)	10

Операций:	2706
Доп. ячеек:	30

Вектор начал работу с ёмкостью 4 и расширился до 32 за 3 удвоения. Результат совпадает со всеми предыдущими тестами.

4.4. Тест 4: несуществующий файл

Файл `input.txt` предварительно удалён. Программа выводит меню, получает выбор и обнаруживает отсутствие файла:

```
Ваш выбор: 1

Чтение файла 'input.txt'...
Ошибка: не удалось открыть файл 'input.txt'
```

Программа корректно обрабатывает ошибку открытия файла и завершает работу с кодом `EXIT_FAILURE`.

4.5. Тест 5: пустой файл

Файл `input.txt` предварительно очищен (содержит только пробел или пуст):

```
Ваш выбор: 1

Чтение файла 'input.txt'...
Ошибка: файл 'input.txt' пуст или не содержит чисел
```

Программа корректно обнаруживает пустой файл и завершает работу с кодом `EXIT_FAILURE`.

5. Блок-схемы алгоритма

Блок-схемы всех подпрограмм приведены в Приложении А (стр. 25).
Перечень блок-схем:

1. Главная программа (`main`)
2. Чтение из файла (`read_file`)
3. Цифровая сортировка (`radix_sort`)
4. Сортировка подсчётом по разряду (`counting_sort_by_digit`)
5. Добавление элемента в вектор (`vector_push`)

6. Исходный код программы

Программа состоит из трёх файлов: `main.c` — точка входа и меню; `radix_sort.h` — объявления всех типов и функций; `radix_sort.c` — реализация структур хранения, ввода/вывода и сортировки.

6.1. Файл `main.c`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #include "radix_sort.h"
5
6 #define INPUT_FILE "input.txt"
7
8 /* Вывести меню и вернуть выбор пользователя (1, 2 или 3). */
9 static int show_menu(void)
10 {
11     int choice = 0; /* +1 присваивание */
12
13     printf("\n\n");
14     printf("    Цифровая сортировка (Radix Sort)    \n");
15     printf("\n");
16     printf("    Выберите структуру данных:          \n");
17     printf("    1. Статический массив                \n");
18     printf("    2. Динамический массив              \n");
19     printf("    3. Вектор (авторасширяемый)         \n");
20     printf("\n");
21     printf("Ваш выбор: ");
22
23     /* Цикл до тех пор, пока не введут корректное значение */
24     while (scanf("%d", &choice) != 1 || choice < 1 || choice > 3) { /* +1 вызов, +3 сравн
25         ↪ ения */
26         int c;
27         while ((c = getchar()) != '\n' && c != EOF); /* очистка stdin */
28         printf("Неверный ввод. Введите 1, 2 или 3: ");
29     }
30
31     return choice; /* +1 выход */
32 }
33
34 /* Итого (корректный ввод с первой попытки): 6 операций
35    (1 присв. + 1 вызов + 3 сравн. + 1 выход) */
36
37 /* Получить указатель на данные и размер из выбранной структуры. */
38 static void get_array_ptr(int mode,
39                             StaticArray *sa, DynamicArray *da, Vector *v,
```

```

38         int **arr_out, int *n_out)
39 {
40     if (mode == 1) {          /* +1 сравнение */
41         *arr_out = sa->data;   /* +2 ссылки (->data), +1 разым. (*arr_out), +1 присваиван
↪ ue */
42         *n_out = sa->size;     /* +2 ссылки (->size), +1 разым. (*n_out), +1 присваиван
↪ ue */
43     } else if (mode == 2) {   /* +1 сравнение */
44         *arr_out = da->data;   /* +2 ссылки (->data), +1 разым. (*arr_out), +1 присваиван
↪ ue */
45         *n_out = da->size;     /* +2 ссылки (->size), +1 разым. (*n_out), +1 присваиван
↪ ue */
46     } else {
47         *arr_out = v->data;    /* +2 ссылки (->data), +1 разым. (*arr_out), +1 присваиван
↪ ue */
48         *n_out = v->size;     /* +2 ссылки (->size), +1 разым. (*n_out), +1 присваиван
↪ ue */
49     }
50 }
51 /* Итого: 10 операций (mode==1: 1 сравн. + 4 + 4 + 1 выход)
52           11 операций (mode==2: 1+1 сравн. + 4 + 4 + 1 выход)
53           11 операций (mode==3: 1+1 сравн. + 4 + 4 + 1 выход) */
54
55 /* Освободить ресурсы выбранной структуры. */
56 static void cleanup(int mode, DynamicArray *da, Vector *v)
57 {
58     if (mode == 2) dynamic_free(da); /* +1 сравнение[, +1 вызов если истина] */
59     if (mode == 3) vector_free(v);  /* +1 сравнение[, +1 вызов если истина] */
60 }
61 /* Итого: 3 операции (mode==1: 2 сравн. + 1 выход)
62           4 операции (mode==2 или 3: 2 сравн. + 1 вызов + 1 выход) */
63
64 int main(void)
65 {
66     int mode = show_menu(); /* +1 вызов, +1 присваивание */
67
68     StaticArray sa;
69     DynamicArray da;
70     Vector v;
71     static_init(&sa); /* +1 вызов */
72     dynamic_init(&da); /* +1 вызов */
73     vector_init(&v);  /* +1 вызов */
74
75     printf("\nЧтение файла '%s'...\n", INPUT_FILE);
76     int n = read_file(INPUT_FILE, mode, &sa, &da, &v); /* +1 вызов, +1 присваивание */
77     if (n <= 0) { /* +1 сравнение */
78         cleanup(mode, &da, &v);
79         return EXIT_FAILURE;

```

```

80     }
81     printf("Прочитано %d чисел.\n", n);
82
83     int *arr = NULL; /* +1 присваивание */
84     int  sz  = 0;    /* +1 присваивание */
85     get_array_ptr(mode, &sa, &da, &v, &arr, &sz); /* +1 вызов */
86
87     printf("\nИсходный массив:\n");
88     print_array(arr, sz); /* O(n) */
89
90     printf("\nВыполняем Radix Sort...\n");
91     radix_sort(arr, sz); /* O(d*(n+k)) */
92
93     printf("Отсортированный массив:\n");
94     print_array(arr, sz); /* O(n) */
95
96     /* Вычисляем d количество разрядов максимального числа */
97     int max_val = find_max(arr, sz); /* +1 вызов, +1 присваивание */
98     int d      = 0; /* +1 присваивание */
99     int tmp = (max_val == 0) ? 1 : max_val; /* +1 сравнение, +1 присваивание */
100    while (tmp > 0) { d++; tmp /= 10; } /* d итераций */
101
102    int ops = 6*sz + 6 + d*(26*sz + 125);
103    printf("\n Оценка сложности \n");
104    printf("  n (элементов)           %d\n", sz);
105    printf("  max элемент                %d\n", max_val);
106    printf("  d (разрядов)                 %d\n", d);
107    printf("  k (основание, BASE)         10\n");
108    printf("  Операций:                    %d\n", ops);
109    printf("  Доп. ячеек:                  %d\n", sz + 10);
110    printf("\n");
111
112    cleanup(mode, &da, &v); /* +1 вызов */
113    return EXIT_SUCCESS; /* +1 возврат */
114 }

```

main: ≈ 20 операций + вызовы функций

6.2. Файл radix_sort.h

```

1 #ifndef RADIX_SORT_H
2 #define RADIX_SORT_H
3
4 #include <stdlib.h>
5
6 /* Статический массив */
7 #define STATIC_MAX 1000

```

```

8
9 typedef struct {
10     int data[STATIC_MAX]; /* фиксированный буфер */
11     int size;              /* фактическое число элементов */
12 } StaticArray;
13
14 /* Динамический массив */
15 typedef struct {
16     int *data; /* указатель на heap-память */
17     int size; /* фактическое число элементов */
18 } DynamicArray;
19
20 /* Вектор (авторасширяемый) */
21 typedef struct {
22     int *data; /* указатель на heap-память */
23     int size; /* фактическое число элементов */
24     int capacity; /* выделенная ёмкость */
25 } Vector;
26
27 /* --- Инициализация --- */
28 void static_init(StaticArray *sa);
29 void dynamic_init(DynamicArray *da);
30 void vector_init(Vector *v);
31
32 /* --- Добавление элемента --- */
33 int static_push(StaticArray *sa, int val);
34 void dynamic_push(DynamicArray *da, int val, int n);
35 void vector_push(Vector *v, int val);
36
37 /* --- Освобождение памяти --- */
38 void dynamic_free(DynamicArray *da);
39 void vector_free(Vector *v);
40
41 /*
42  * Цифровая сортировка (Radix Sort, LSD от младшего разряда к старшему).
43  *
44  * Принимает массив arr из n целых неотрицательных чисел,
45  * сортирует его на месте (результат остаётся в arr).
46  *
47  * Сложность:
48  *   Время  $O(d \cdot (n + k))$  где d количество цифр в max-элементе,
49  *   k основание (10 для десятичной),
50  *   n число элементов.
51  *   Память  $O(n + k)$  вспомогательный массив + счётчики.
52  *
53  * При фиксированных d и k сложность линейная:  $O(n)$ .
54  */
55 void radix_sort(int *arr, int n);

```



```

56
57 /* Вспомогательная функция: найти максимальный элемент в arr[0..n-1] */
58 int find_max(const int *arr, int n);
59
60 /*
61  * Считать числа из файла filename.
62  * Способ хранения выбирается параметром mode:
63  *   1   статический массив
64  *   2   динамический массив
65  *   3   вектор
66  *
67  * Заполняет переданные структуры (только одну, соответствующую mode).
68  * Возвращает количество прочитанных чисел или -1 при ошибке.
69  */
70 int read_file(const char *filename, int mode,
71               StaticArray *sa, DynamicArray *da, Vector *v);
72
73 /* Вывести массив arr из n элементов на экран. */
74 void print_array(const int *arr, int n);
75
76 #endif /* RADIX_SORT_H */

```

6.3. Файл radix_sort.c

Статический массив

```

1 /*
2  * СТАТИЧЕСКИЙ МАССИВ
3  * Память выделяется на стеке. Размер ограничен STATIC_MAX.
4  * */
5
6 void static_init(StaticArray *sa)
7 {
8     sa->size = 0; /* +2 ссылки (->size), +1 присваивание */
9 }
10 /* Итого: 4 операции (2 ссылки + 1 присв. + 1 выход) */
11
12 /* Добавить val в конец. Возвращает 0 при успехе, -1 при переполнении. */
13 int static_push(StaticArray *sa, int val)
14 {
15     if (sa->size >= STATIC_MAX) { /* +2 ссылки (->size), +1 сравнение */
16         fprintf(stderr, "Ошибка: статический массив переполнен (макс. %d)\n",
17                 STATIC_MAX); /* +1 вызов */
18         return -1; /* +1 выход */
19     }
20     sa->data[sa->size] = val; /* +2 ссылки (->data), +2 ссылки (->size), +1 индексация,
    ↪ +1 присваивание */

```

```

21     sa->size++;          /* +2 ссылки (->size), +1 арифметика, +1 присваивание */
22     return 0;           /* +1 выход */
23 }
24 /* Итого (норм. путь): 14 операций */

```

static_init: 4 операции (2 ссылки + 1 присв. + 1 выход)

static_push: 14 операций (норм. путь: 2 ссылки + 1 сравн. + 4 ссылки + 1 инд. + 1 присв. + 2 ссылки + 1 арифм. + 1 присв. + 1 выход)

Динамический массив

```

1  /*
2  *  ДИНАМИЧЕСКИЙ МАССИВ
3  *  Выделяем ровно n элементов через malloc без realloc.
4  *  */
5
6  void dynamic_init(DynamicArray *da)
7  {
8      da->data = NULL; /* +2 ссылки (->data), +1 присваивание */
9      da->size = 0;    /* +2 ссылки (->size), +1 присваивание */
10 }
11 /* Итого: 7 операций (+ выход из метода) */
12
13 /* Выделить память для n элементов. */
14 static void dynamic_alloc(DynamicArray *da, int n)
15 {
16     da->data = (int *)malloc(n * sizeof(int)); /* +1 арифметика, +1 вызов, +2 ссылки (->
    ↪ data), +1 присваивание */
17     if (!da->data) { /* +2 ссылки (->data), +1 сравнение */
18         fprintf(stderr, "Ошибка: не удалось выделить память\n"); /* +1 вызов */
19         exit(EXIT_FAILURE); /* +1 вызов */
20     }
21 }
22 /* Итого (норм. путь): 9 операций (+ выход из метода) */
23
24 /* Записать val в конец. При первом вызове выделяет память. */
25 void dynamic_push(DynamicArray *da, int val, int n)
26 {
27     if (da->data == NULL) /* +2 ссылки (->data), +1 сравнение */
28         dynamic_alloc(da, n); /* +1 вызов */
29     da->data[da->size] = val; /* +2 ссылки (->data), +2 ссылки (->size), +1 индексация,
    ↪ +1 присваивание */
30     da->size++; /* +2 ссылки (->size), +1 арифметика, +1 присваивание */
31 }
32 /* Итого (норм. путь, без вызова alloc): 14 операций (+ выход из метода) */
33

```

```

34 void dynamic_free(DynamicArray *da)
35 {
36     free(da->data); /* +2 ссылки (->data), +1 вызов */
37     da->data = NULL; /* +2 ссылки (->data), +1 присваивание */
38     da->size = 0; /* +2 ссылки (->size), +1 присваивание */
39 }
40 /* Итого: 10 операций (+ выход из метода) */

```

dynamic_init: 7 операций (4 ссылки + 2 присв. + 1 выход)

dynamic_alloc: 9 операций (норм. путь: 1 арифм. + 1 вызов + 2 ссылки + 1 присв. + 2 ссылки + 1 сравн. + 1 выход)

dynamic_push: 14 операций (норм. путь: 2 ссылки + 1 сравн. + 4 ссылки + 1 инд. + 1 присв. + 2 ссылки + 1 арифм. + 1 присв. + 1 выход)

dynamic_free: 10 операций (2 ссылки + 1 вызов + 2 ссылки + 1 присв. + 2 ссылки + 1 присв. + 1 выход)

Вектор

```

1  /*
2   * ВЕКТОР (авторасширяемый динамический массив)
3   * Начальная ёмкость = 4. При заполнении удваивается.
4   * Стратегия удвоения: O(1) амортизированного времени на push.
5   * */
6
7  #define VECTOR_INIT_CAP 4
8
9  void vector_init(Vector *v)
10 {
11     v->data = NULL; /* +2 ссылки (->data), +1 присваивание */
12     v->size = 0; /* +2 ссылки (->size), +1 присваивание */
13     v->capacity = 0; /* +2 ссылки (->capacity), +1 присваивание */
14 }
15 /* Итого: 10 операций (+ выход из метода) */
16
17 /* Добавить val в конец вектора, расширив при необходимости. */
18 void vector_push(Vector *v, int val)
19 {
20     if (v->size >= v->capacity) { /* +2 ссылки (->size), +2 ссылки (->
    ↪ capacity), +1 сравнение */
21         int new_cap = (v->capacity == 0) /* +2 ссылки (->capacity), +1 сравнение
    ↪ */
22             ? VECTOR_INIT_CAP
23             : v->capacity * 2; /* +2 ссылки (->capacity), +1 арифметика
    ↪ */

```

```

24                                     /* +1 присваивание (new_cap =) */
25     int *tmp = (int *)realloc(v->data, /* +2 ссылки (->data) */
26                                     new_cap * sizeof(int)); /* +1 арифметика */
27                                     /* +1 вызов, +1 присваивание (tmp =) */
28     if (!tmp) { /* +1 сравнение */
29         fprintf(stderr, "Ошибка: realloc вернул NULL\n"); /* +1 вызов */
30         exit(EXIT_FAILURE); /* +1 вызов */
31     }
32     v->data = tmp; /* +2 ссылки (->data), +1 присваивание */
33     v->capacity = new_cap; /* +2 ссылки (->capacity), +1 присваивание */
34 }
35 v->data[v->size] = val; /* +2 ссылки (->data), +2 ссылки (->size), +1 индексация, +1
↪ присваивание */
36 v->size++; /* +2 ссылки (->size), +1 арифметика, +1 присваивание */
37 }
38 /* Итого: 16 операций (без realloc); 35 операций (с realloc, не первый вызов);
39     32 операции (с realloc, первый вызов capacity==0, нет умножения) */
40
41 void vector_free(Vector *v)
42 {
43     free(v->data); /* +2 ссылки (->data), +1 вызов */
44     v->data = NULL; /* +2 ссылки (->data), +1 присваивание */
45     v->size = 0; /* +2 ссылки (->size), +1 присваивание */
46     v->capacity = 0; /* +2 ссылки (->capacity), +1 присваивание */
47 }
48 /* Итого: 13 операций (+ выход из метода) */

```

vector_init: 10 операций (6 ссылок + 3 присв. + 1 выход)

vector_push: 16 операций (без realloc); 35 операций (с realloc, не первый); 32 операции (с realloc, первый вызов)

vector_free: 13 операций (8 ссылок + 1 вызов + 3 присв. + 1 выход)

Ввод/вывод

```

1  /*
2   * ВВОД / ВЫВОД
3   * */
4
5  /*
6   * read_file считывание чисел из текстового файла.
7   * Два прохода: первый считает n, второй заполняет структуру.
8   * Итого: 10n + 12 операций (не считая внутренних push-функций).
9   */
10 int read_file(const char *filename, int mode,
11               StaticArray *sa, DynamicArray *da, Vector *v)
12 {

```

```

13 FILE *fp = fopen(filename, "r"); /* +1 вызов, +1 присваивание */
14 if (!fp) { /* +1 сравнение */
15     fprintf(stderr, "Ошибка: не удалось открыть файл '%s'\n", filename);
16     return -1;
17 }
18
19 /* Первый проход: подсчёт чисел */
20 int n = 0; /* +1 присваивание */
21 int tmp;
22 while (fscanf(fp, "%d", &tmp) == 1) /* n+1 вызовов, n+1 сравнений */
23     n++; /* +1 арифметика */
24
25 if (n == 0) { /* +1 сравнение */
26     fprintf(stderr, "Ошибка: файл '%s' пуст или не содержит чисел\n", filename);
27     fclose(fp);
28     return -1;
29 }
30
31 rewind(fp); /* +1 вызов */
32
33 /* Второй проход: i=0(+1 присв.); i<n(n+1 сравн.); i++(n арифм.) */
34 for (int i = 0; i < n; i++) {
35     int val;
36     if (fscanf(fp, "%d", &val) != 1) /* +1 вызов, +1 сравнение */
37         break;
38
39     if (mode == 1) { /* +1 сравнение */
40         if (static_push(sa, val) != 0) { fclose(fp); return -1; }
41     } else if (mode == 2) { /* +1 сравнение */
42         dynamic_push(da, val, n);
43     } else {
44         vector_push(v, val);
45     } /* +1 вызов (push) */
46 }
47
48 fclose(fp); /* +1 вызов */
49 return n; /* +1 выход */
50 }
51 /* Итого: 10n + 12 операций */
52
53 /*
54  * print_array вывод элементов массива через пробел.
55  * Итого: 7n + 3 операций.
56  */
57 void print_array(const int *arr, int n)
58 {
59     /* i=0: +1 присв.; i<n: n+1 сравн.; i++: n арифм. */
60     for (int i = 0; i < n; i++) {

```

```

61     printf("%d", arr[i]); /* +1 индексация, +1 вызов */
62     if (i < n - 1)        /* +1 арифметика, +1 сравнение */
63         printf(" ");      /* +1 вызов */
64 }
65 printf("\n"); /* +1 вызов */
66 }
67 /* Итого: 7n + 3 операций */

```

read_file: $10n + 12$ операций (не считая внутренности push-функций)

print_array: $7n + 3$ операций

Цифровая сортировка

```

1  /*
2  *  RADIX SORT
3  *  */
4
5  #define BASE 10
6
7  /* find_max поиск максимального элемента в arr[0..n-1].
8  * Итого: 6n 1 операций (худший случай). */
9  int find_max(const int *arr, int n)
10 {
11     int max = arr[0]; /* +1 индексация, +1 присваивание */
12     /* i=1: +1 присв.; i<n: n сравн.; i++: n-1 арифм. */
13     for (int i = 1; i < n; i++) {
14         if (arr[i] > max) /* +1 индексация, +1 сравнение */
15             max = arr[i]; /* +1 индексация, +1 присваивание */
16     }
17     return max; /* +1 выход из метода */
18 }
19 /* Итого: 6n 1 операций (худший случай) */
20
21 /*
22 *  counting_sort_by_digit устойчивая сортировка подсчётом
23 *  по одному разряду exp (1, 10, 100, ).
24 *  Обход справа налево на шаге 4 гарантирует устойчивость.
25 *
26 *  Подсчёт (k = BASE = 10):
27 *      Подготовка (malloc + проверка) : 4
28 *      Шаг 1 (обнуление count[0..k-1]) : 4k + 2 = 42
29 *      Шаг 2 (подсчёт вхождений)      : 8n + 2
30 *      Шаг 3 (префиксные суммы)        : 7k 5 = 65
31 *      Шаг 4 (расстановка в output[])  : 13n + 3
32 *      Шаг 5 (копирование обратно)     : 5n + 2
33 *      free + выход                    : 2

```

```

34  *
35  *   Итого:  $26n + 11k + 10 = 26n + 120$  (при  $k=10$ )
36  */
37  static void counting_sort_by_digit(int *arr, int n, int exp)
38  {
39      int *output = (int *)malloc(n * sizeof(int)); /* +1 арифметика, +1 вызов, +1 присваивание
    ↪ ание */
40      if (!output) { /* +1 сравнение */
41          fprintf(stderr, "Ошибка: не удалось выделить память для output\n");
42          exit(EXIT_FAILURE);
43      }
44
45      int count[BASE];
46
47      /* Шаг 1: i=0(+1 присв.); i<k: k+1 сравн.; i++: k арифм.;
48         count[i](+1 индекс.), =0(+1 присв.) k раз
49         Итого шаг 1:  $4k + 2 = 42$  */
50      for (int i = 0; i < BASE; i++)
51          count[i] = 0; /* +1 индексация, +1 присваивание */
52
53      /* Шаг 2: i=0(+1 присв.); i<n: n+1 сравн.; i++: n арифм.;
54         arr[i](+1 индекс.), /exp(+1 арифм.), %BASE(+1 арифм.),
55         digit=(+1 присв.), count[digit](+1 индекс.), ++(+1 арифм.) n раз
56         Итого шаг 2:  $8n + 2$  */
57      for (int i = 0; i < n; i++) {
58          int digit = (arr[i] / exp) % BASE; /* +1 индексация, +1 арифметика, +1 арифметика
    ↪ , +1 присваивание */
59          count[digit]++; /* +1 индексация, +1 арифметика */
60      }
61
62      /* Шаг 3: i=1(+1 присв.); i<k: k сравн.; i++: k-1 арифм.;
63         count[i](+1 индекс.), i-1(+1 арифм.), count[i-1](+1 индекс.),
64         +=(+1 арифм.+1 присв.) (k-1) раз
65         Итого шаг 3:  $7k + 5 = 65$  */
66      for (int i = 1; i < BASE; i++)
67          count[i] += count[i - 1]; /* +1 индексация, +1 арифметика, +1 индексация, +1 ариф
    ↪ метика, +1 присваивание */
68
69      /* Шаг 4: i=n-1(+1 арифм.+1 присв.); i>=0: n+1 сравн.; i--: n арифм.;
70         arr[i](+1 инд.), /exp(+1 арифм.), %BASE(+1 арифм.), digit=(+1 присв.),
71         count[digit](+1 инд.), -1(+1 арифм.), output[...](+1 инд.),
72         arr[i](+1 инд.), =(+1 присв.), count[digit](+1 инд.), --(+1 арифм.) n раз
73         Итого шаг 4:  $13n + 3$  */
74      for (int i = n - 1; i >= 0; i--) {
75          int digit = (arr[i] / exp) % BASE; /* +1 индексация, +1 арифметика, +1 арифметик
    ↪ а, +1 присваивание */
76          output[count[digit] - 1] = arr[i]; /* +1 индексация, +1 арифметика, +1 индексаци
    ↪ я, +1 индексация, +1 присваивание */

```

```

77         count[digit]--;                                /* +1 индексация, +1 арифметика */
78     }
79
80     /* Шаг 5: i=0(+1 присв.); i<n: n+1 сравн.; i++: n арифм.;
81         arr[i](+1 инд.), output[i](+1 инд.), =(+1 присв.)  n раз
82         Итого шаг 5: 5n + 2 */
83     for (int i = 0; i < n; i++)
84         arr[i] = output[i]; /* +1 индексация, +1 индексация, +1 присваивание */
85
86     free(output); /* +1 вызов */
87 }
88 /* Итого: 26n + 11k + 10 операций (k=10: 26n + 120) */
89
90 /*
91  * radix_sort  цифровая сортировка LSD.
92  * Выполняет d проходов counting_sort_by_digit по разрядам
93  * exp = 1, 10, 100, пока max / exp > 0.
94  */
95 void radix_sort(int *arr, int n)
96 {
97     if (n <= 1) return; /* +1 сравнение, +1 выход */
98
99     int max_val = find_max(arr, n); /* +1 вызов, +1 присваивание */
100
101     /* exp=1(+1 присв.); max/exp(+1 арифм.) >0(+1 сравн.): d+1 раз;
102        exp*=BASE(+1 арифм.+1 присв.): d раз;
103        вызов counting_sort: d раз */
104     for (int exp = 1; max_val / exp > 0; exp *= BASE)
105         counting_sort_by_digit(arr, n, exp);
106 }
107 /* Итого (собственный код radix_sort): 7 + 5d операций
108     Итого полностью (с подфункциями):
109     6n + 6 + d(26n + 11k + 15) = 6n + 6 + d(26n + 125) при k=10 */

```

find_max: $6n - 1$ операций (худший случай)

counting_sort_by_digit: $26n + 11k + 10$ операций ($k=10$: $26n + 120$)

radix_sort: $6n + 6 + d \cdot (26n + 11k + 15)$ операций итого ($k=10$:
 $6n + 6 + d \cdot (26n + 125)$)

Заключение

В ходе выполнения лабораторной работы была разработана программа на языке C, реализующая цифровую сортировку (Radix Sort LSD) целых неотрицательных чисел, считываемых из файла.

Программа организована в три файла: `main.c` содержит точку входа и меню; `radix_sort.h` объявляет все типы и функции; `radix_sort.c` реализует три структуры хранения, ввод/вывод и алгоритм сортировки.

Пользователю предоставляется выбор из трёх структур хранения: статического массива (фиксированный буфер на стеке), динамического массива (однократный `malloc` после подсчёта элементов) и вектора (авторасширяемый массив с удвоением ёмкости).

Теоретическая сложность алгоритма составляет $O(d \cdot (n + k))$ по времени и $O(n + k)$ по памяти. При фиксированных d и k алгоритм работает за **линейное** время $O(n)$. Результаты тестирования подтверждают корректность сортировки для всех трёх структур данных, а также правильную обработку ошибочных ситуаций.

Приложение А. Блок-схемы алгоритма

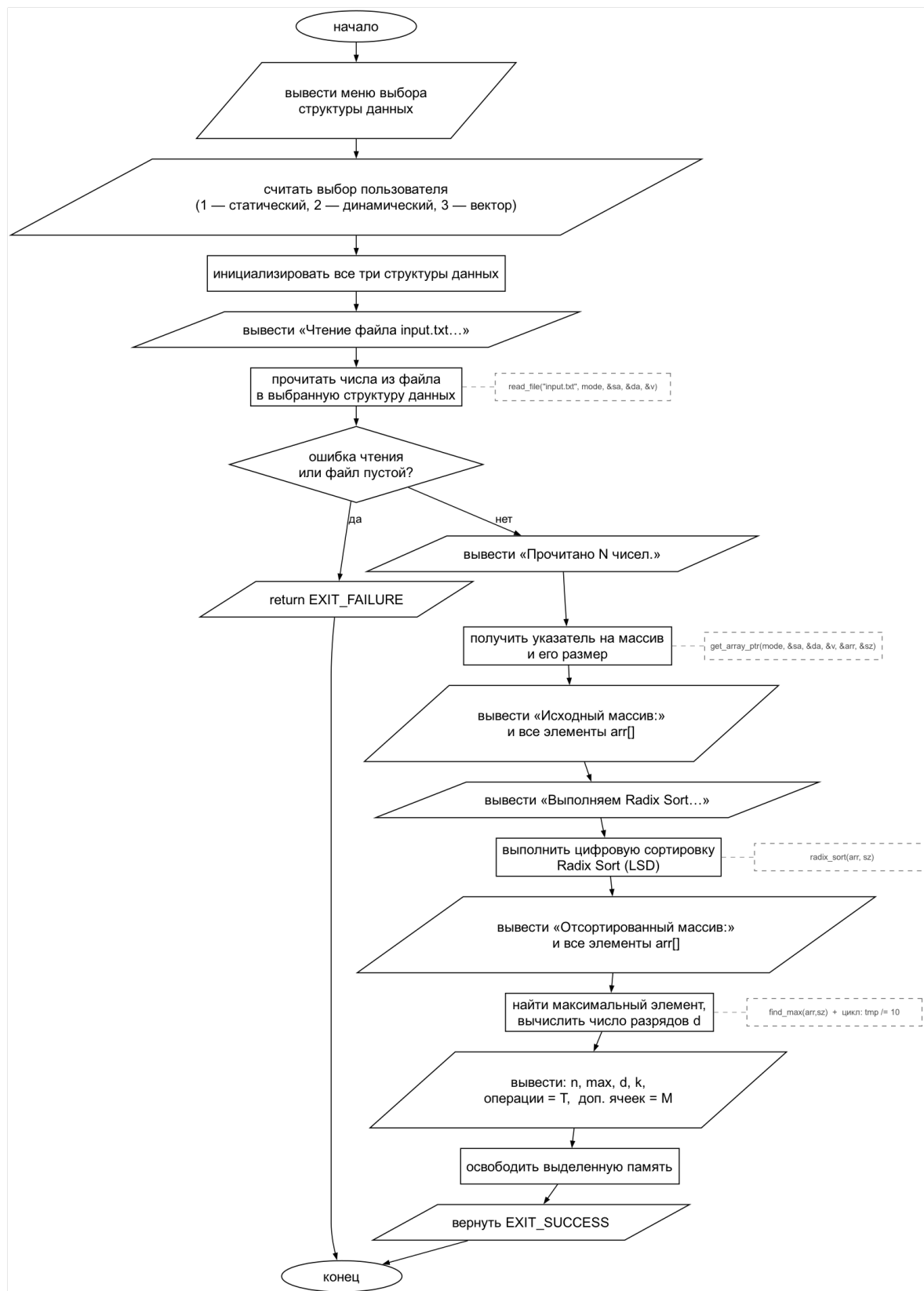


Рис. 1 – Главная программа (main)

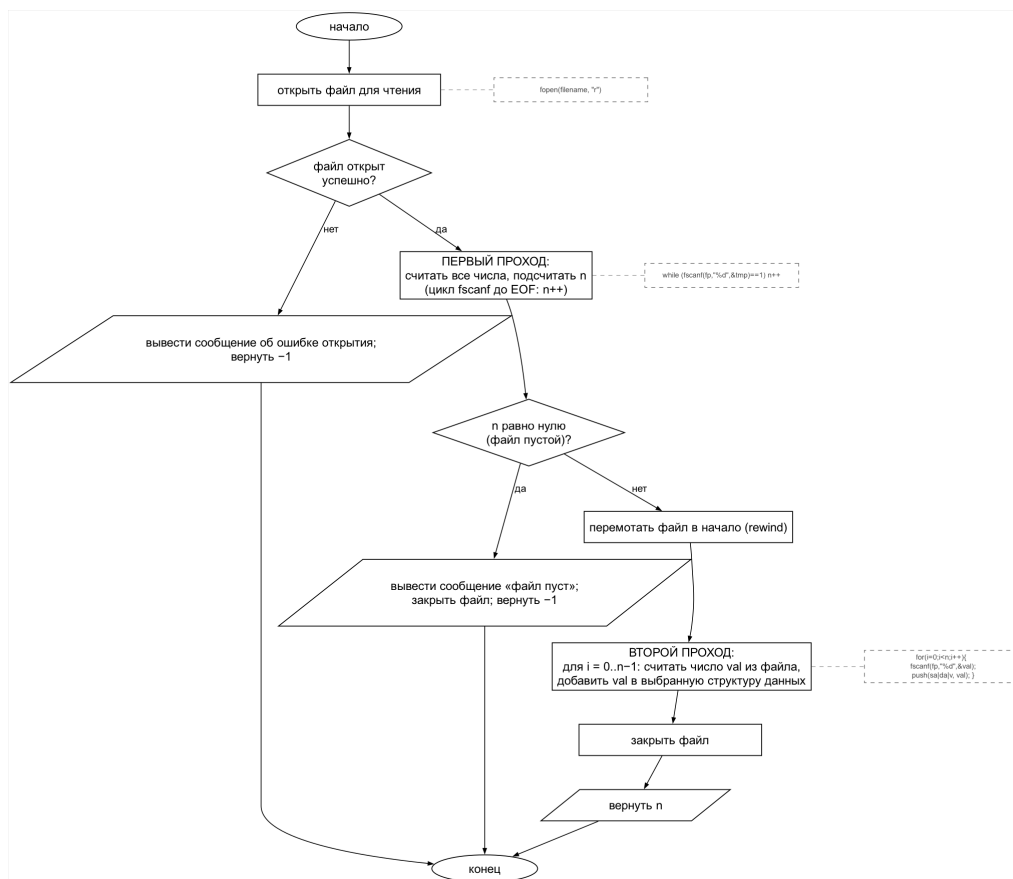


Рис. 2 – Чтение из файла (read_file)

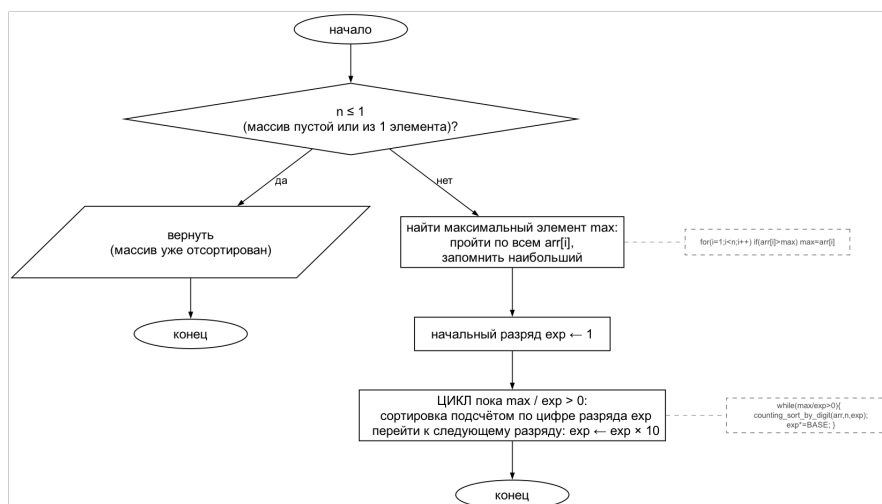


Рис. 3 – Цифровая сортировка (radix_sort)

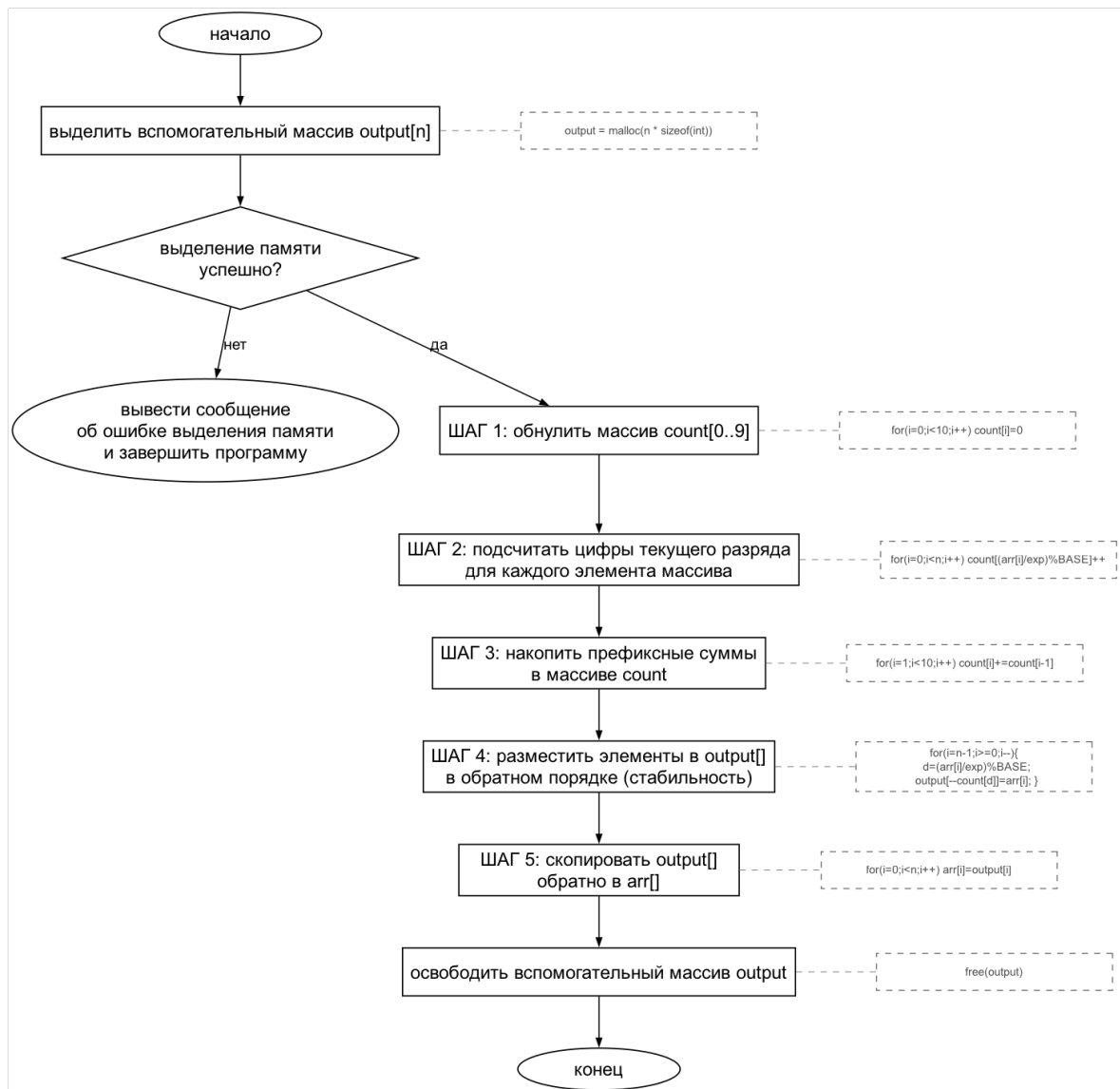


Рис. 4 – Сортировка подсчётом по разряду (counting_sort_by_digit)

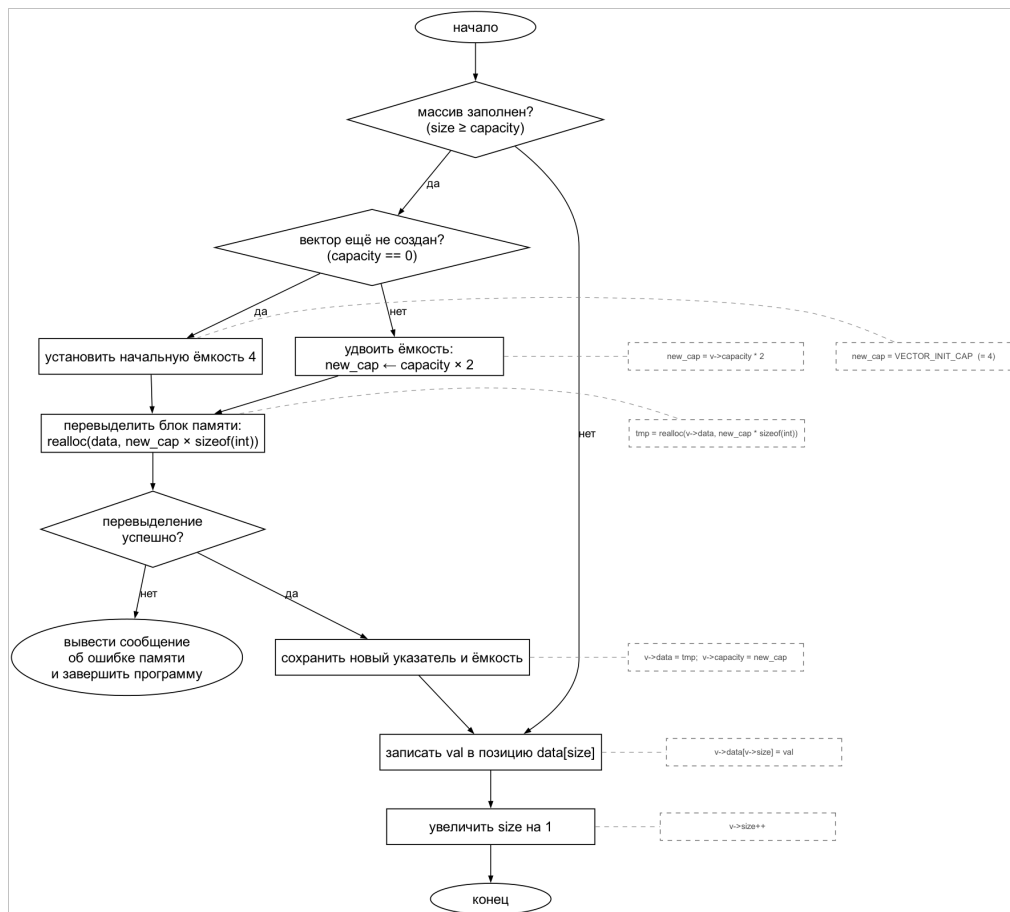


Рис. 5 – Добавление элемента в вектор (vector_push)